

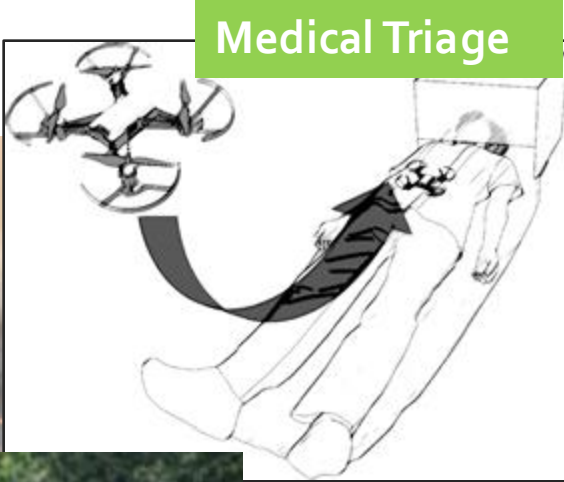
Reinforcement Learning-Driven Edge Management for Reliable Multi-view 3D Reconstruction

*Motahare Mounesan, Sourya Saha, Houchao Gan, Md. Nurul Absur,
Saptarshi Debroy*

Visual Computing Applications For Mission-Critical Use Cases



Disaster Response



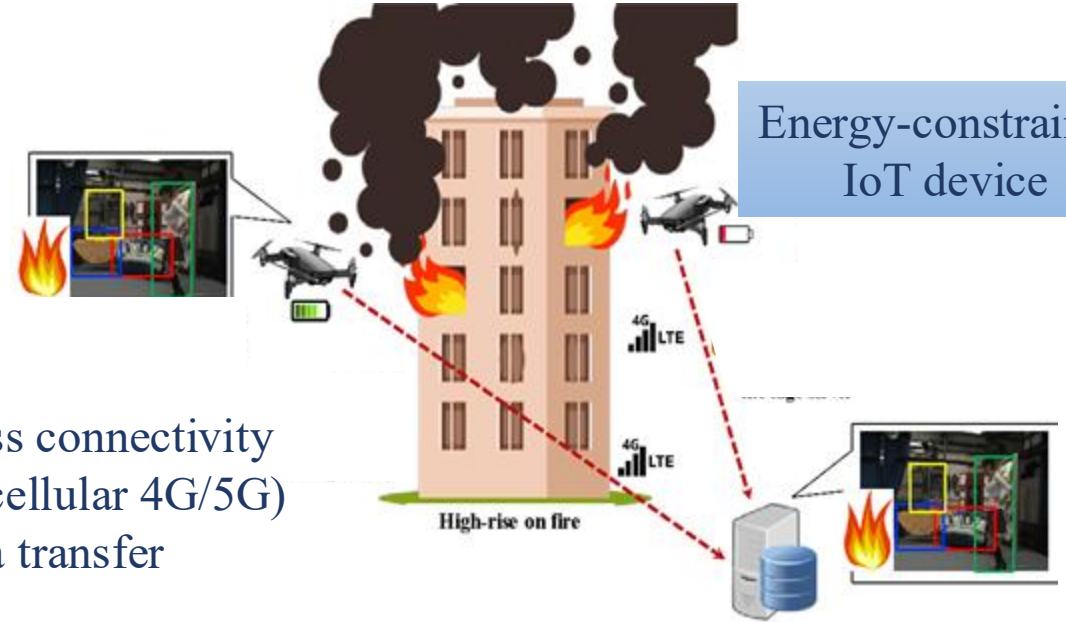
Medical Triage



Tactical Scenarios

Just-in-time edge environment

Strict latency requires **real-time** DNN inference



Energy-constrained IoT device

Wireless connectivity (often cellular 4G/5G) for data transfer

Moderately powerful edge servers

Challenges:

1. Energy-constraint
2. Time-sensitive
3. Unpredictability (ad-hoc + dynamic environment)

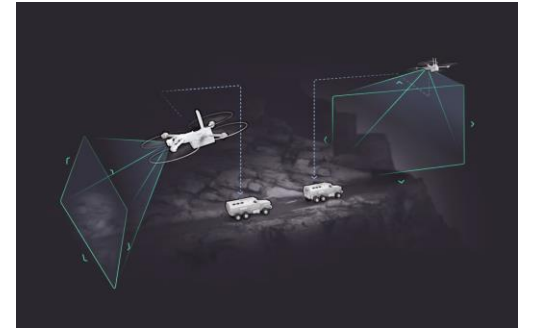
Specific Use Case: Multi-view 3D Reconstruction

Provides a unified, spatially accurate view that

1. Integrates scattered data sources
2. Reveals occluded hazards, clarifies depth and structure
→ situational awareness
3. Supports quick, confident decision-making when every second counts.



Disaster Response



Tactical Scenarios

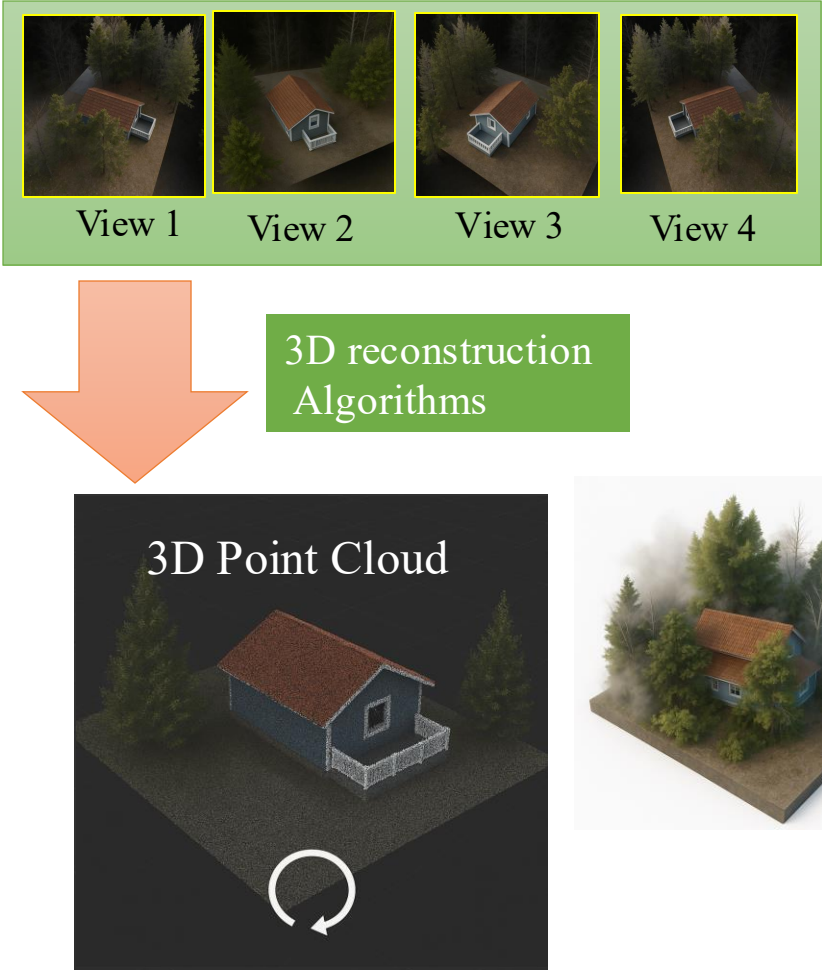
Just-in-time edge environment

Time-sensitive

Multi-view 3D Reconstruction in Mission Critical applications



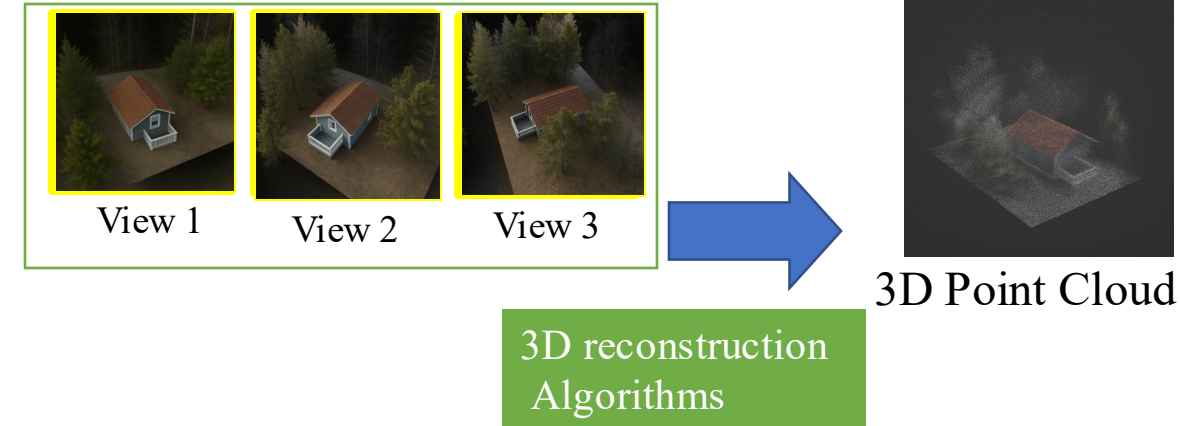
UAVs are wirelessly connected to base stations (LTE/5G)



Generic Visual Computing vs Multi-view 3D Reconstruction

	Aspect	Generic Applications	3D Reconstruction
1	Processing Model	Run independently	Relies on coordinated multi-device input
2	On-Device Utility	Benefit from on-device processing	Requires centralized or collaborative processing
3	Computational Load	Relatively lighter	Computation-intensive
4	Scene Characteristics	Handle static or slowly changing data	Deal with dynamic, time-sensitive scenes
5	Input Requirements	May work with basic input	Demand high-quality, synchronized data
6	Robustness to Disruptions	Tolerate minor disruptions	Highly sensitive to sensor failures, occlusions, and network delays

Time-sensitivity – System-Level challenges

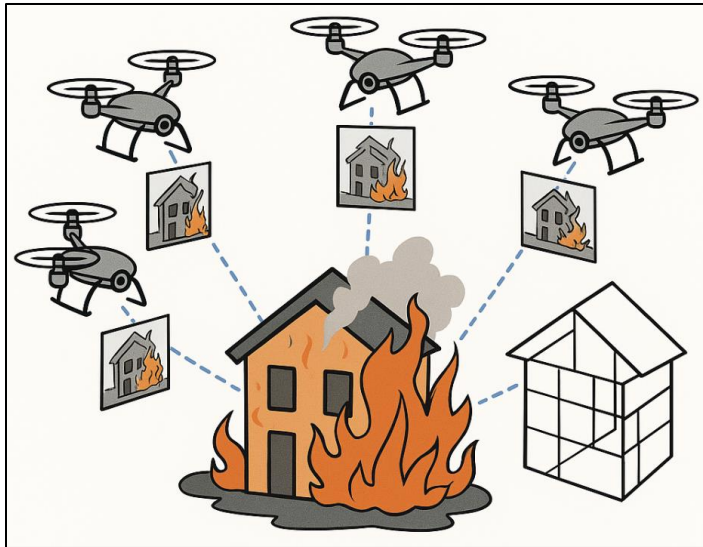


- **Dynamic Scene**
Requires rapid processing to keep up with **real-time** scene

Computational Load vs. View Redundancy:
More camera views enhance quality but increase processing time — selective view usage is key for time-sensitive performance.

UAVs are wirelessly connected to base stations (LTE/5G)

Reconstruction Quality – Application specific vulnerabilities in Dynamic Scenes



- **Reconstruction Quality Depends On:**
 - **Clean, consistent input** → sensitive to noise, occlusion, misalignment
 - **Synchronized multi-view input** → missing or delayed views degrade output
- **Application Demand:**
 - Operates in **fast-changing environments** → **No tolerance for delay or retries**



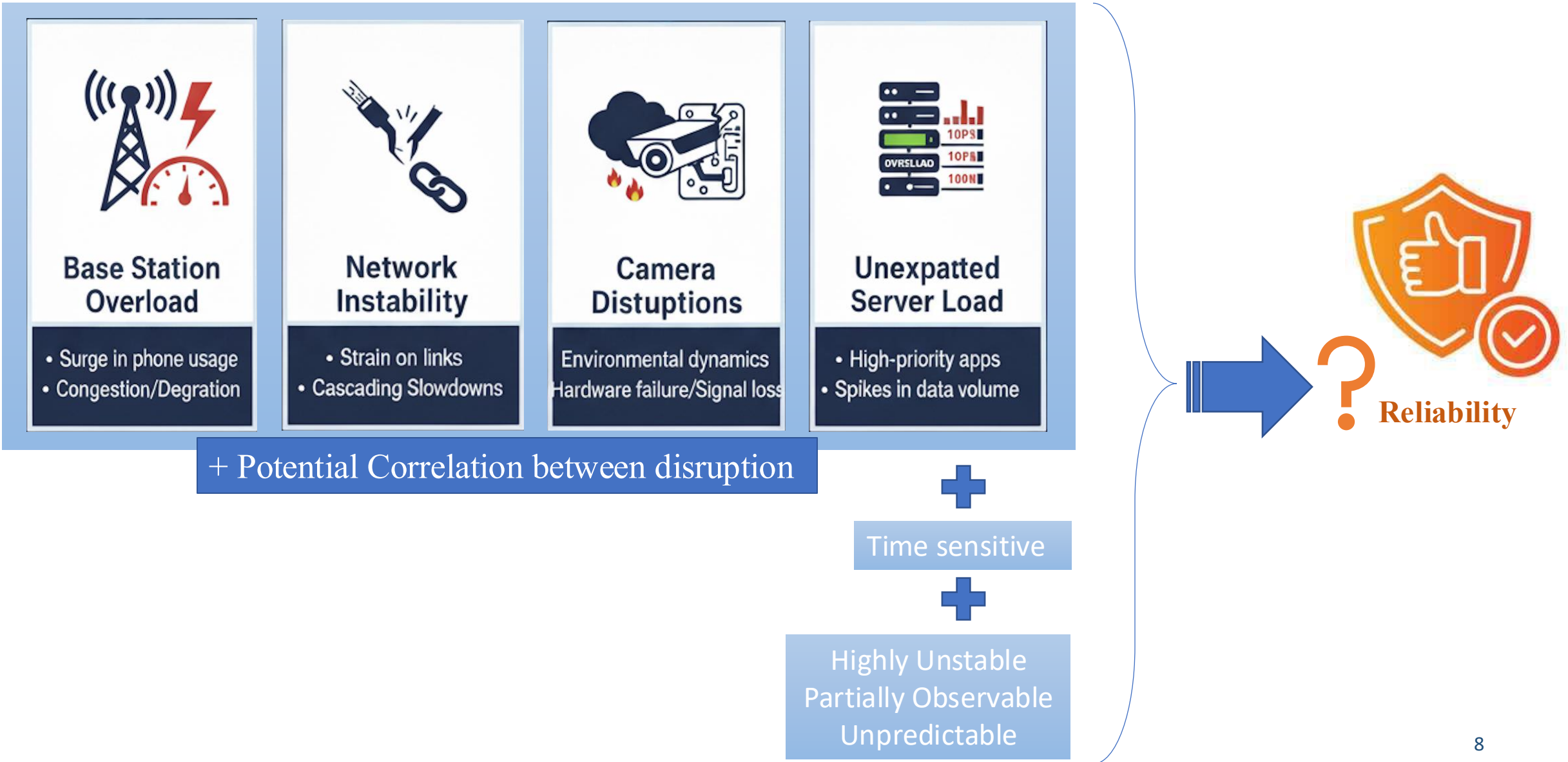
Natural blocking



Occlusion



Potential Sources of Disruptions



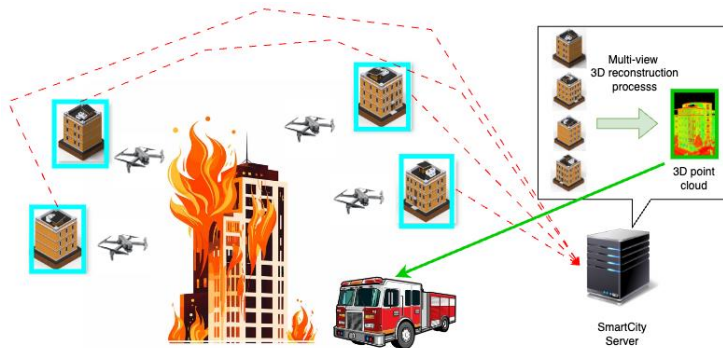
Problem Definition

Camera-Equipped UAV/ IoT Device

Homogeneous/heterogeneous
Prone to disruptions

End Servers

Smart city edge servers
Homogeneous/heterogeneous
computational/load



Edge Controller

A **centralized decision-maker** that ideally selects:

- **Camera views** to use
 - **Image resolution**
 - **Edge server** for processing
 - **Network path** for transmission
- etc.

All with the goal of delivering **reliable 3D reconstruction**—meeting **quality requirements** within **latency constraints** under dynamic, partially observable, disruption-prone conditions.

Partial observability, Non-stationary conditions

+ **Delayed feedback**

Adaptive, real-time decisions under uncertainty:



Reinforcement Learning (RL)

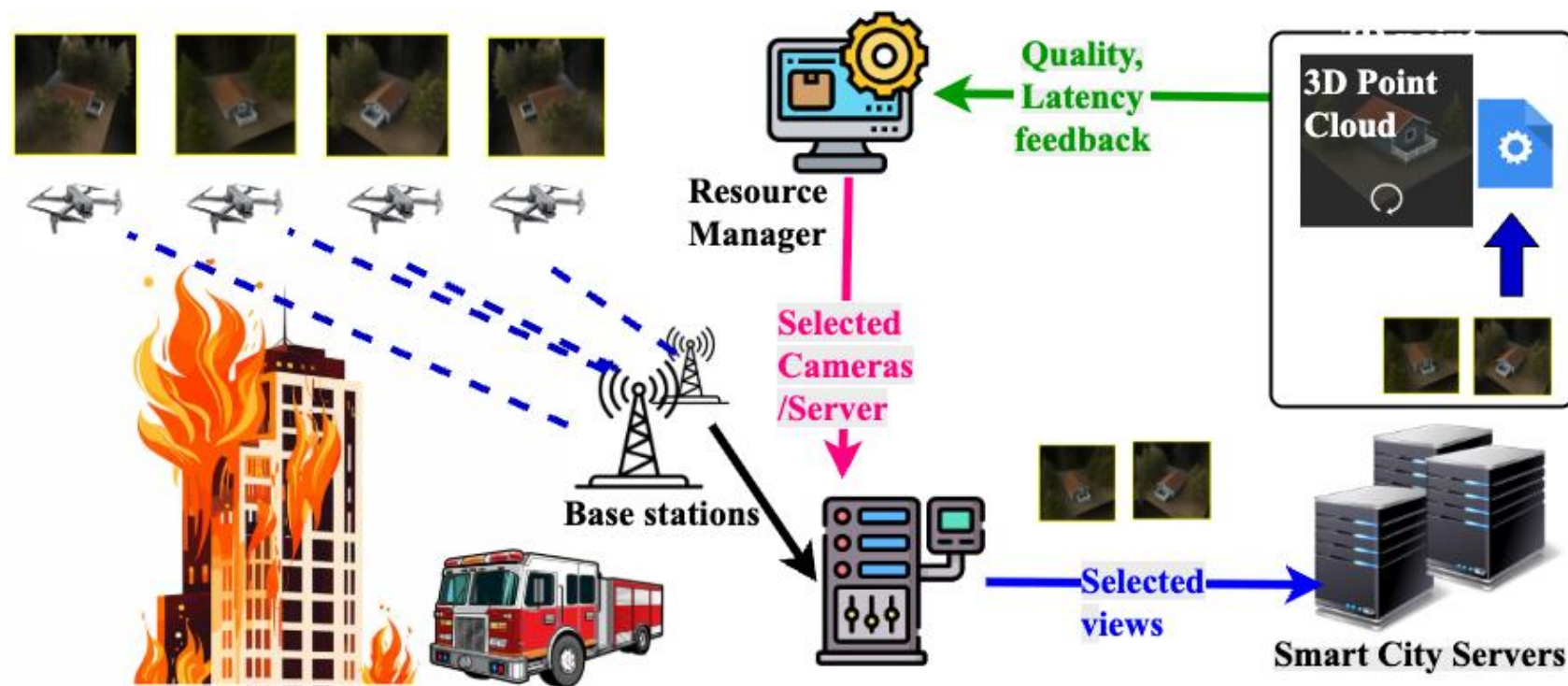
Interacting (act, feedback), adapt

System Model

At each time step, the system selects:

- **Subset of cameras**
- **Edge server** for processing

It then **observes**: Point cloud quality, Reconstruction latency, End-to-end latency



Objective: Maximize reliability, defined as the number of time steps where the reconstruction quality meets a **minimum threshold** and the end-to-end latency stays within a **predefined bound**.

Problem Formulation

We formulate the problem as a Markov Decision Processes, and define two cooperative agents:

Reliability aware reward design:

$$\text{Reliability} = \begin{cases} 1, & \text{if } PQ_t \geq \Theta_{PQ_{\min}} \text{ and } Latency_{total} \leq \Theta_{L_{\max}} \\ 0, & \text{otherwise} \end{cases}$$

Camera Selection Agent

Stateless MDP

Action: Select a camera subset

Reward:

$$R_t^{\text{camera}} = w_1 \cdot S_Q^{(t)} + w_2 \cdot S_L^{(t)}$$

$$S_Q^{(t)} = \min \left(1, \frac{PQ_t}{\Theta_{PQ_{\text{ref}}}} \right)$$

$$S_L^{(t)} = \max \left(0, 1 - \frac{Latency_t}{\Theta_{L_{\max}}} \right)$$

Number of
cameras



Server Selection Agent

State:

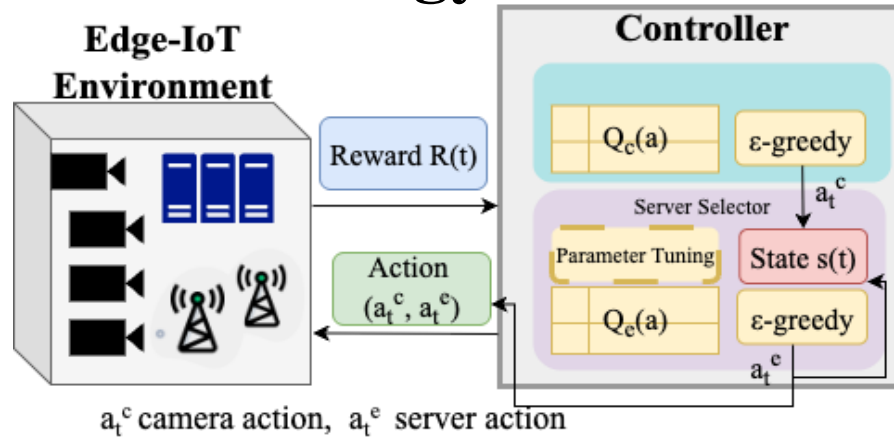
$$s_t = (\# \text{ selected cameras, previous server})$$

Action: Select a server

Reward:

$$R_t^{\text{server}} = \max \left(0, 1 - \frac{Latency_{total}^{(t)}}{\Theta_{L_{\max}}} \right)$$

Solution Strategy



Q-Learning

1. Model-Free Learning

No need for prior knowledge of environment dynamics

2. Lightweight & Efficient

Ideal for **real-time decision-making**

3. Supports Delayed Rewards

4. Adaptability Under Uncertainty

sample efficient

Exploration Strategy:

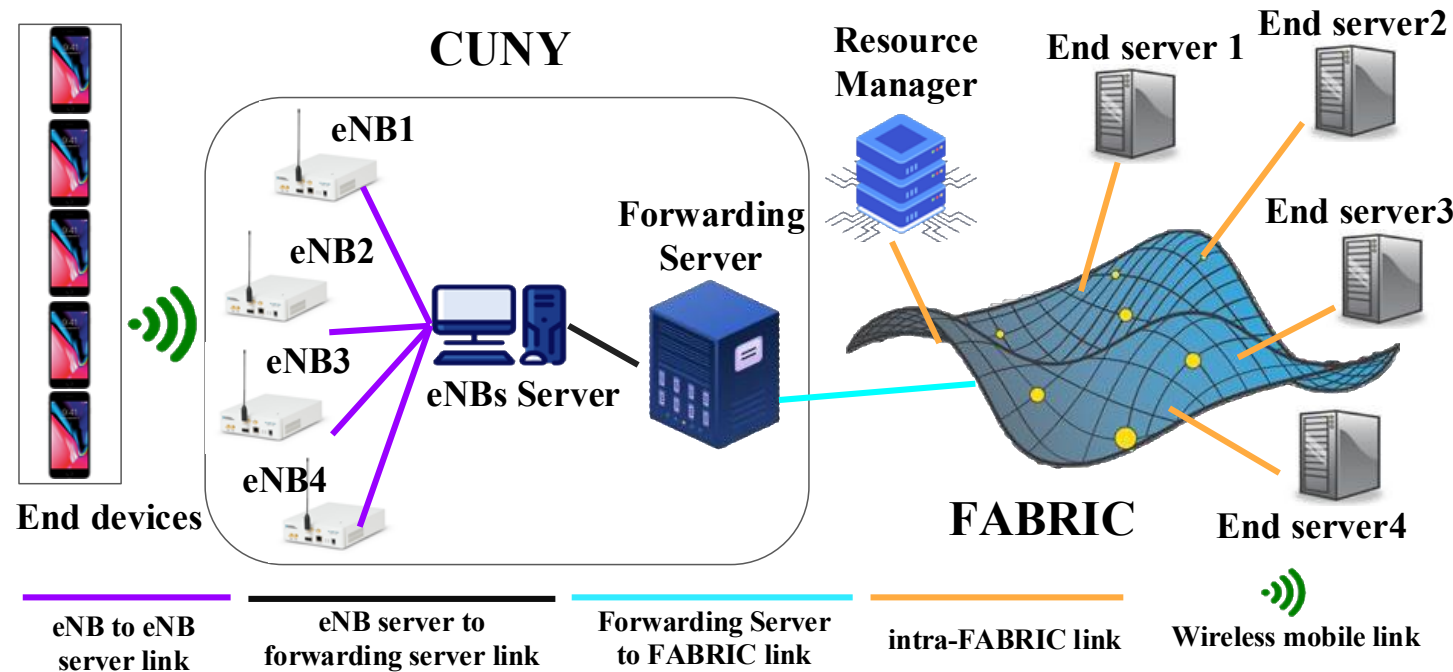
Epsilon-greedy: explore new actions with probability epsilon

Two Q-learning Variants:

- **Standard**
constant exploration rate and learning rate
- **Adaptive**
 - **Stable phase:**
Gradual decay of learning rate and exploration rate for convergence.
 - **Reactive phase:**
Sudden increase in learning rate, exploration rate if performance drops to re-adapt quickly.

Evaluation

Testbed implementation



End Devices:

$5 \times$ *Nokia 2.2* smartphones sending camera with the rate of 1.43 frame per second

Base Stations: eNBs (LTE)

LTE Network built using srsRAN4G and USRP-2901 SDRs

End Servers: Hosted on **FABRIC** testbed (GPU-accelerated)

Hardware Specs:

Resource Manager – 20 cores, 32GB RAM, 100GB Disk, No GPU

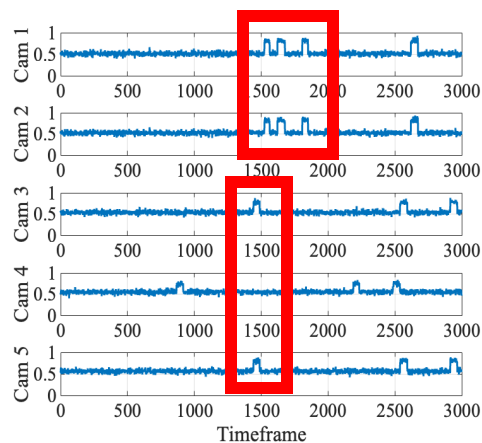
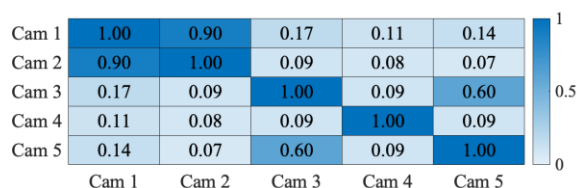
End Server – 16 cores, 32GB RAM, 100GB Disk, RTX 6000 GPU

Evaluation

Experiment Setup/ Dataset

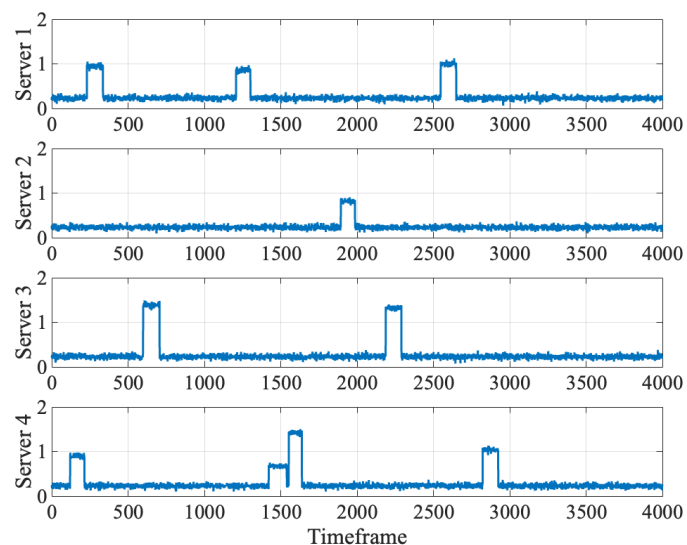
- **Camera-side disruption**

- Inject (5/10) random bump events with 50 to 100 timeframe duration
- Synchronize bumps across camera groups (e.g., Cameras 1 & 2).
- Threshold the time series

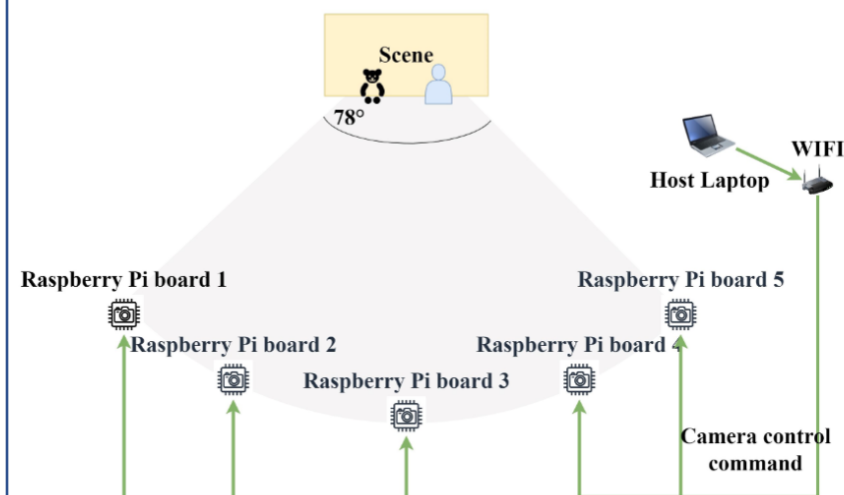


- **Server-side disruption**

Inject random latency spike events, short
Assign each spike to a random server independently



- **3D Multi-view Dataset**



Capture dynamic indoor scenes with 5 synchronized Raspberry Pi cameras
Record long video sequences at 25–30 FPS

Evaluation

Metrics

1. **Reliability Score:** number of frames meeting both latency(1s reconstruction, 1.5 latency) and quality constraints (600 avg matching points)
2. **Latency:** End-to-end delay
3. **3D Reconstruction Quality:**

2D-reprojection:

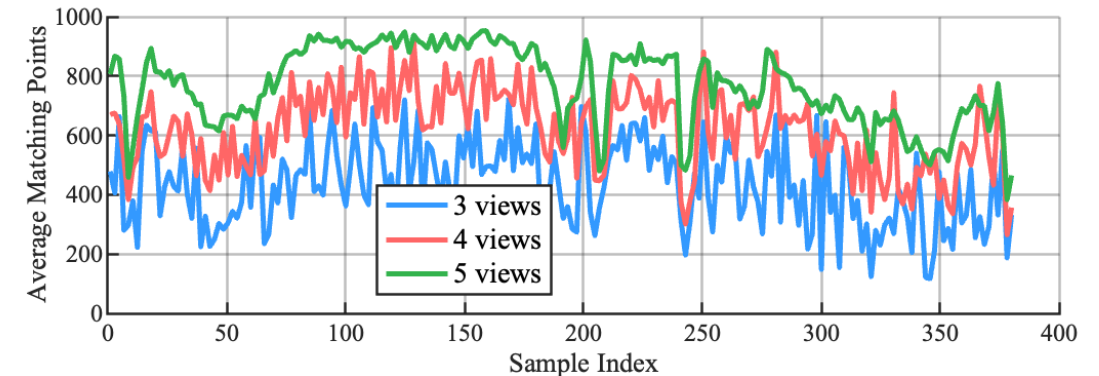
Compare projected point cloud colors with original pixel colors

Perceptual color distance using CompuPhase metric

Models human vision: more sensitive to green

$\Delta E < 20$ = perceptually acceptable

Per-view match: Average number of matching points per view



3D Reconstruction Pipeline

Benchmarked:

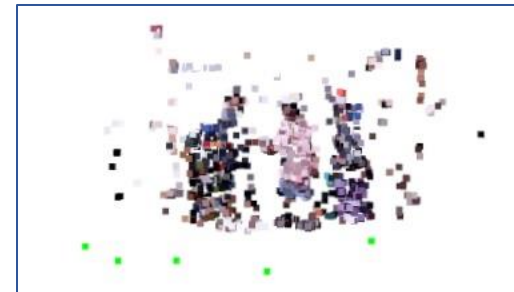
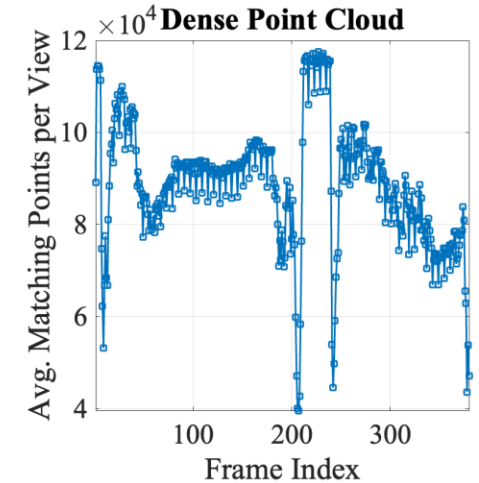
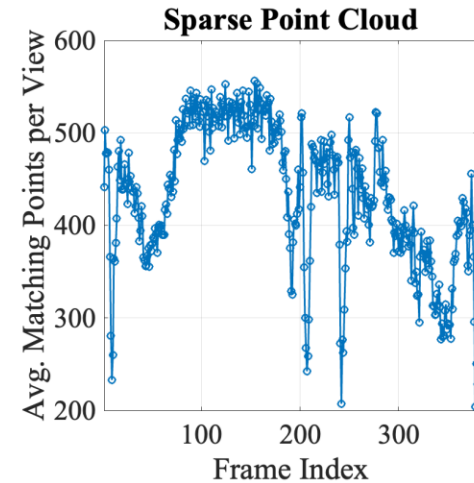
MVSNet (DNN-based):
~20s per scene (5 views)

OpenMVG (geometric):
Structure from Motion
Multi-View Stereo
~8s per scene (5 views)

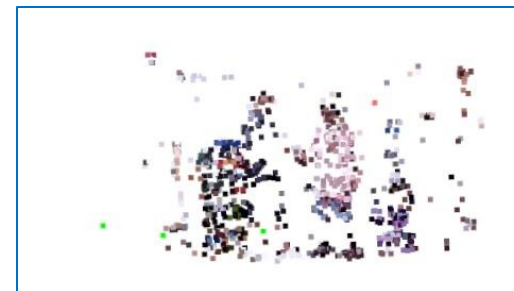
Too much delay for the reward

Sparse point cloud as a alternate:

SfM: Fewer points
Correlates with high dense quality
Effective for fast quality estimation and decision-making



Sparse vs Dense with 5 cameras



Sparse vs Dense with 3 cameras



Evaluation

Baselines

Camera Selection Baselines

1. Greedy-3

Fixed policy → picks the subset with the best average historical performance

2. Epsilon-Greedy Bandit

Stateless learner, using latency + quality as feedback (adapts using $\epsilon = 0.1$, $\alpha = 0.1$)

3. Random

Uniformly samples from all valid subsets

Server Selection Baselines

1. Latency-Greedy

Chooses server with lowest recent latency

2. Round-Robin

Cycles through servers in fixed order (no adaptation)

Camera Agent Ablation Study

- Highly disrupted interval
- Round Robin for steady server selection

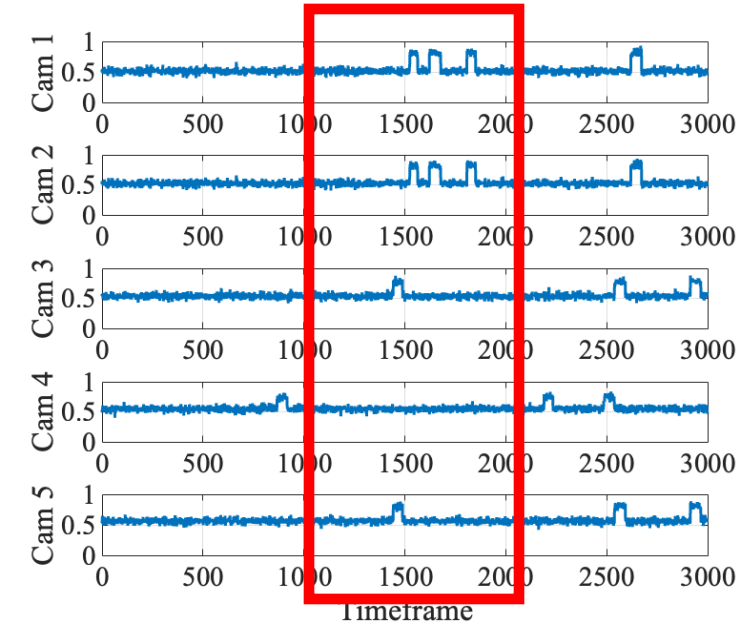
Q-learning

Learns optimal camera subset using fixed learning and exploration rates

Adaptive Q-learning

Adjusts fixed learning and exploration rates based on system performance; reacts to disruptions and recovers faster

Highly unstable

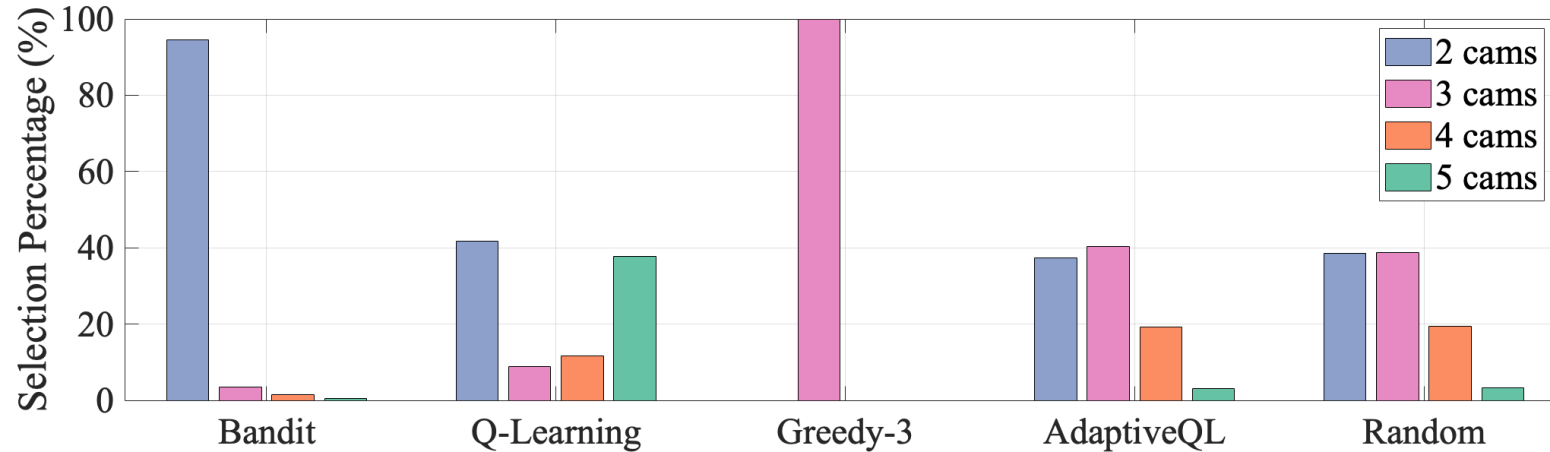


Camera Selector	Avg PO	Std PO	Avg Recon Lat. (s)	Std Recon Lat.	Avg Total Lat. (s)	Std Total Lat.	Reliability
Q-learning	582.80	309.15	0.79	0.36	2.54	0.82	62.53%
Greedy-3	542.90	300.93	0.67	0.36	2.45	0.91	59.92%
Epsilon-Greedy Bandit	443.10	134.67	0.71	0.22	3.45	0.88	24.69%
Adaptive Q-learning	378.16	263.23	0.70	0.33	2.47	0.84	36.38%
Random	386.87	262.79	0.69	0.32	2.26	0.85	25.05%

High exploration rate

Camera Agent Ablation Study

Camera Selection Distribution

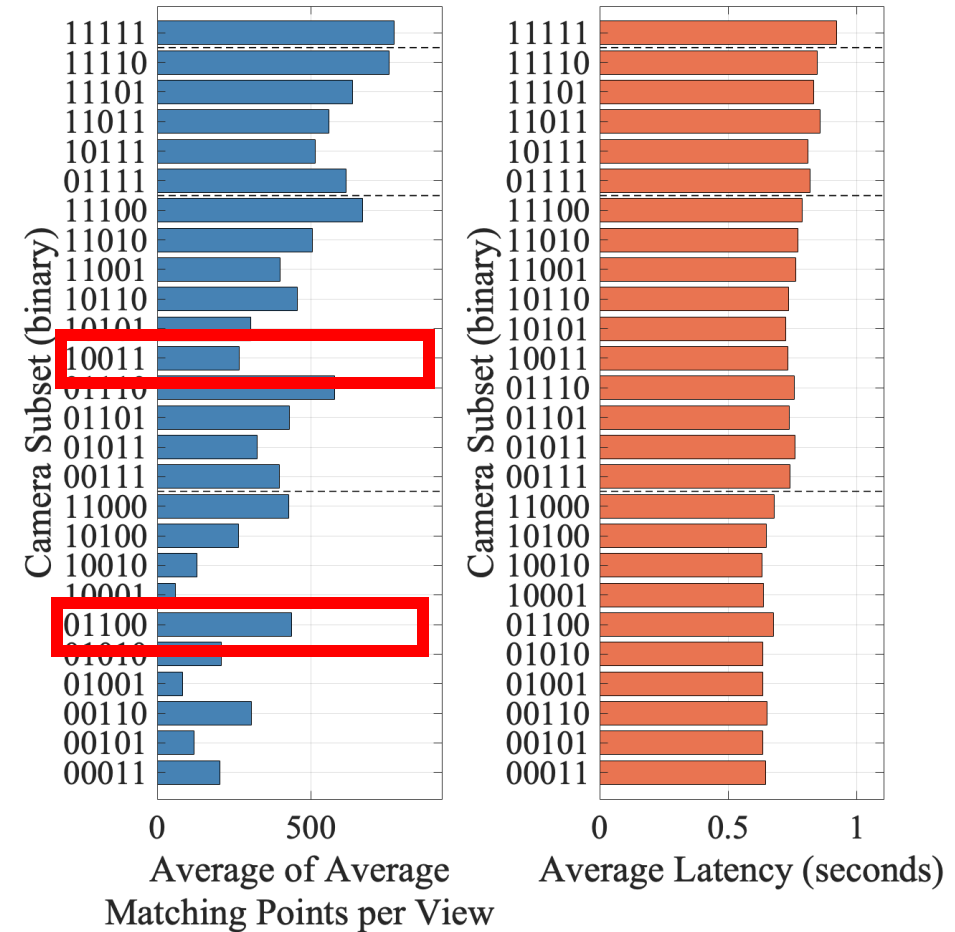
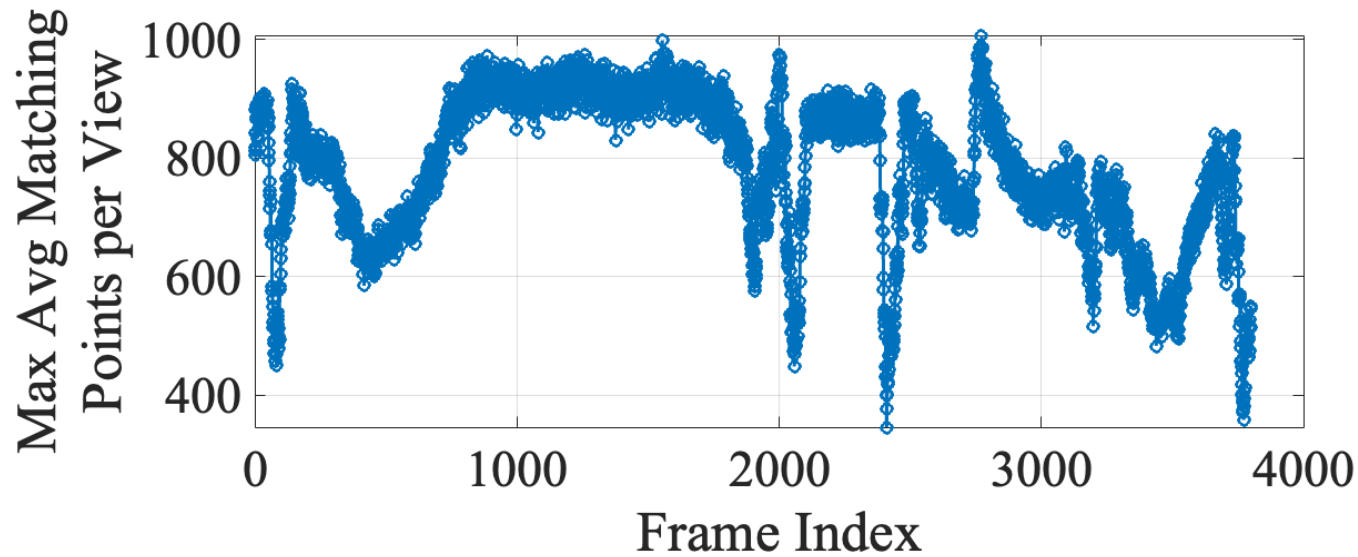


Camera Selector	Avg PO	Std PO	Avg Recon Lat. (s)	Std Recon Lat.	Avg Total Lat. (s)	Std Total Lat.	Reliability
Q-learning	582.80	309.15	0.79	0.36	2.54	0.82	62.53%
Greedy-3	542.90	300.93	0.67	0.36	2.45	0.91	59.92%
Epsilon-Greedy Bandit	443.10	134.67	0.71	0.22	3.45	0.88	24.69%
Adaptive Q-learning	378.16	263.23	0.70	0.33	2.47	0.84	36.38%
Random	386.87	262.79	0.69	0.32	2.26	0.85	25.05%

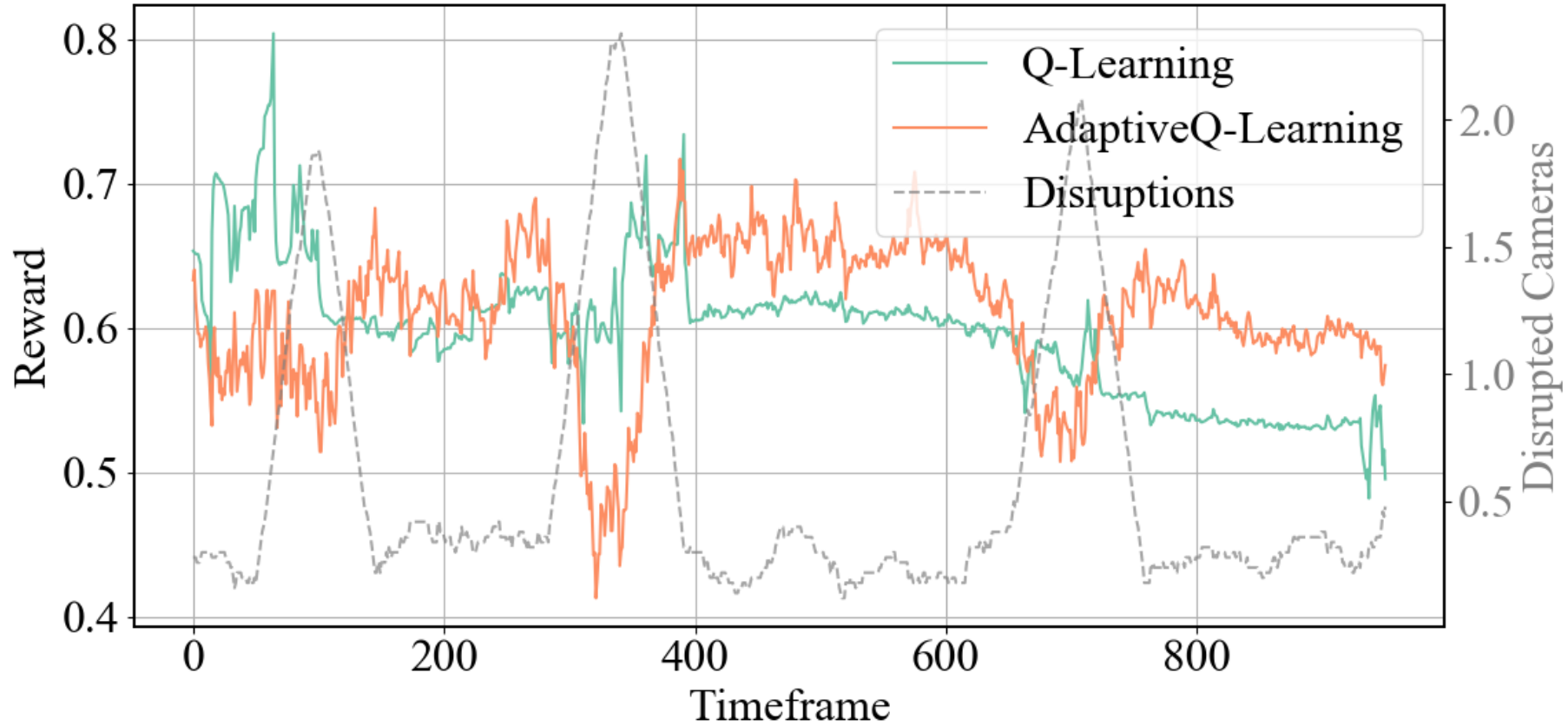
High exploration rate

Camera Agent Data Analysis – in Absence of Disruptions

- Scene dynamics impact reconstruction quality
- More cameras $\neq$ better reconstruction
- Subset selection is scene-dependent



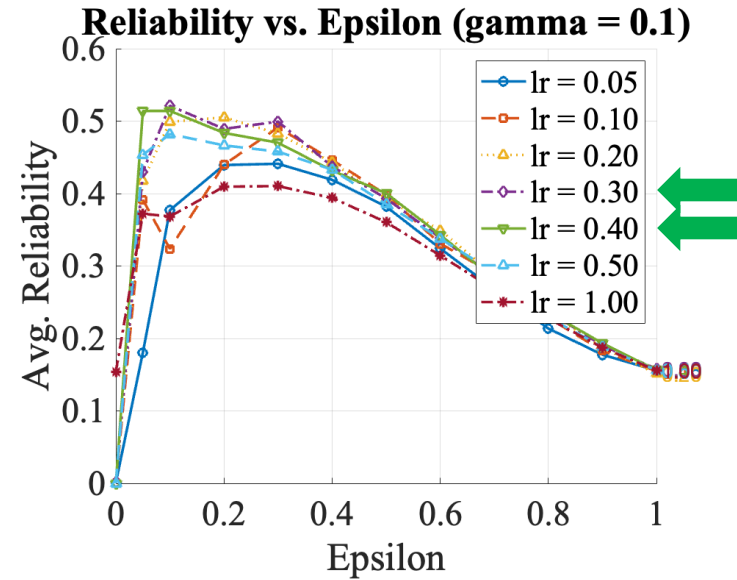
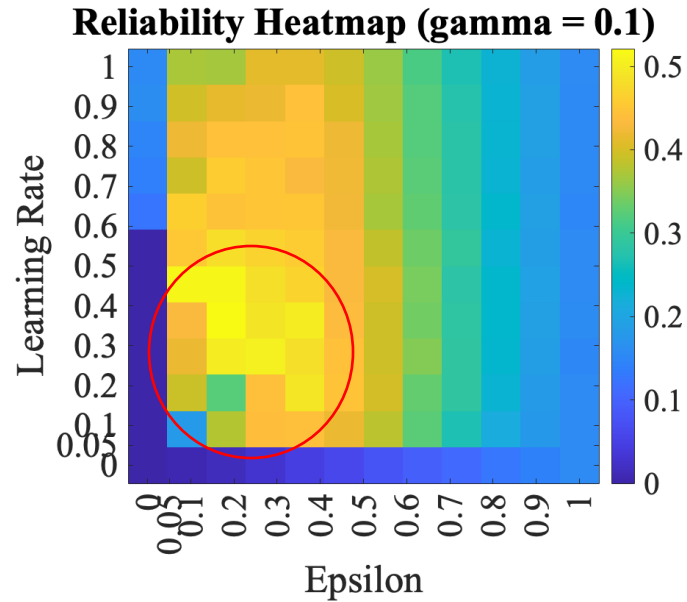
Q-Learning vs Adaptive Q-Learning



Agent	Avg. Reward	Avg. PQ	Avg. Latency (s)	Reliability
Q-Learning	0.59	531.02	0.91	34.8%
Adaptive Q-Learning	0.79	493.89	0.97	31.6%

Q-Learning sensitivity Analysis

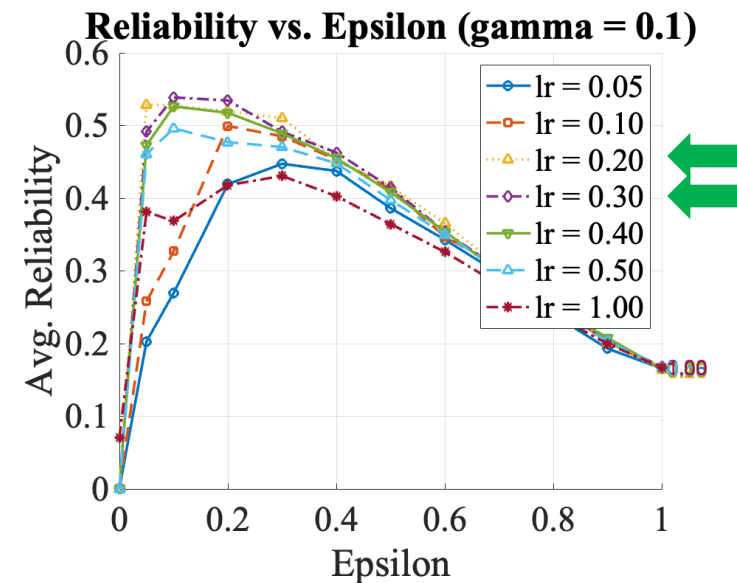
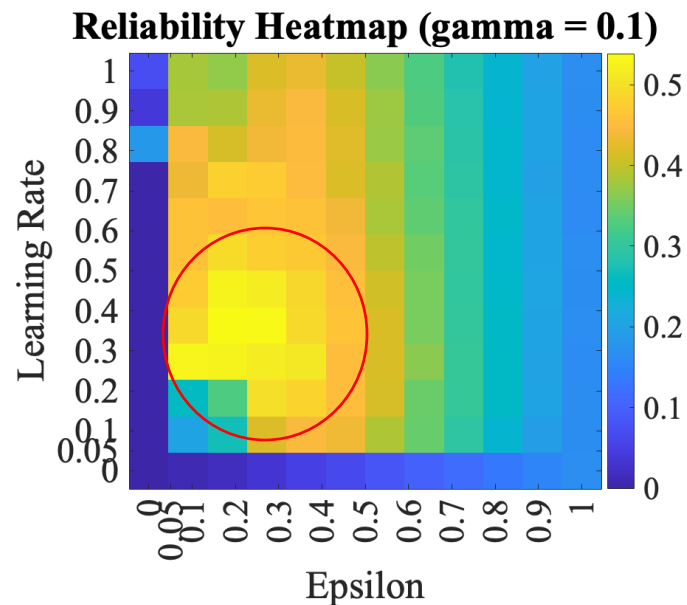
Short term
camera
disruptions



Lower
exploration rate

Low to average
learning rate

Longer
camera
disruptions



Server Agent Ablation Study

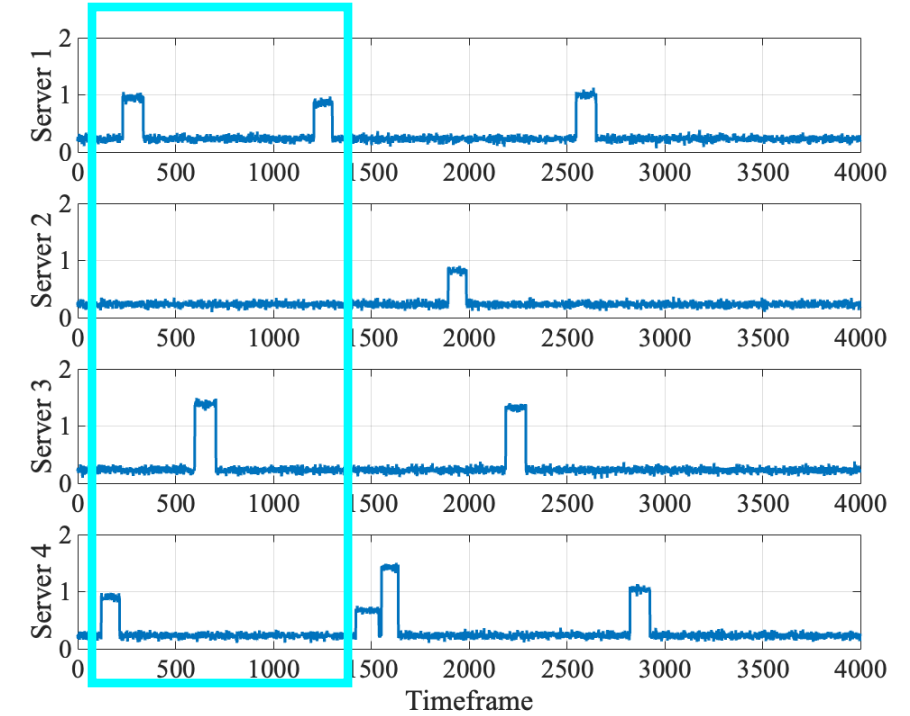
Greedy3 for steady camera selection

Adaptive Q-learning offers best trade-off: responsive + convergent

Round-Robin performs consistently but poorly under disruptions

Q-learning adapts gradually and performs better over time

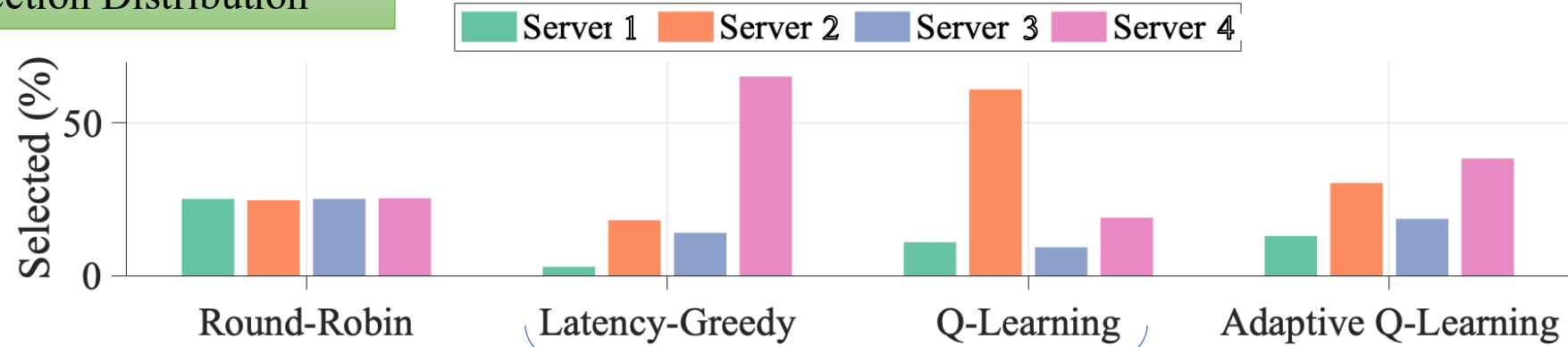
Latency-Greedy is fast-reactive but overloads servers



Server Agent	Total Lat. (s)	Total Lat. Std	Recon Lat. (s)	Recon Lat. Std	Reliability
Round-Robin	3.20	0.70	0.91	0.15	31.5%
Latency-Greedy	5.32	6.99	2.14	1.85	4.1%
O-Learning	3.47	5.43	2.72	2.27	16.9%
Adaptive Q-Learning	3.62	1.89	2.13	1.97	55.0%

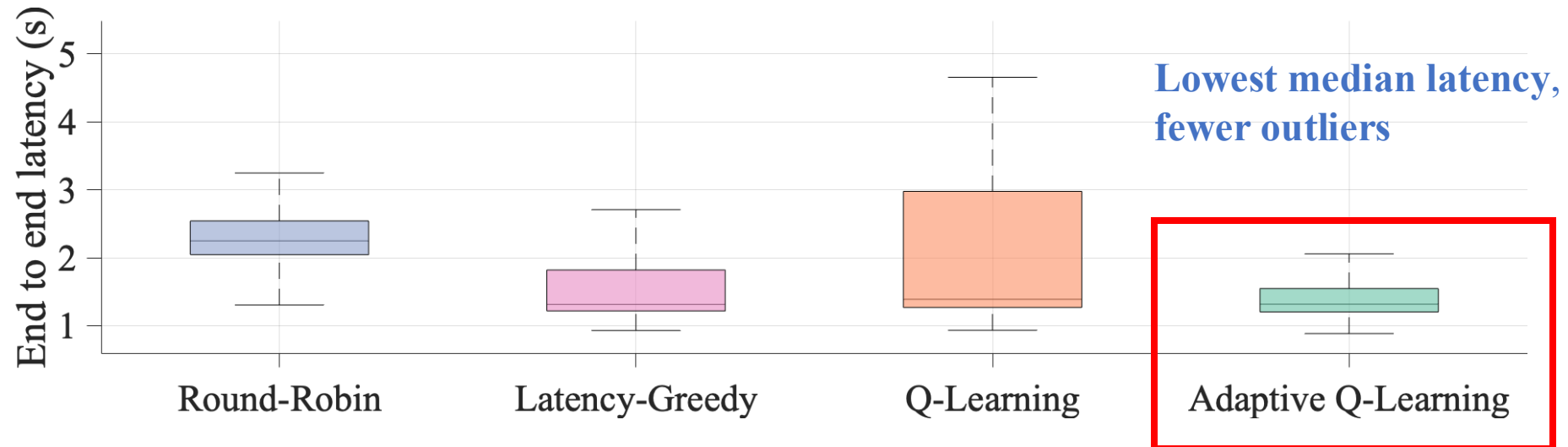
Server Agent Ablation Study

Server Selection Distribution



Server 2 and 4 are more stable

End to End Latency Distribution



Discussion of Results

Aspect	Camera Agent	Server Agent
<i>Decision Space</i>	Large (26 subsets)	Small (4 servers)
<i>Adaptation Speed</i>	Slower	Faster
<i>Behavior in High Disruption</i>	Adaptive Q-learning behaves more randomly; standard Q-learning more stable	Consistent and reliable
<i>Performance</i>	Slower adaptation but still effective	Clear reliability gains
<i>Compared to Baselines</i>	Outperforms bandit and random	Outperforms bandit and random

Conclusions

Impact of Feedback Latency

- Earlier feedback (e.g., sparse point clouds) improves learning dynamics
- Reducing feedback delay can yield further gains

Observed Gains (Highly Unstable Environments)

- **Camera Agent:** +15% reliability over random, +2% over Greedy3
- **Server Agent:** +24% over round-robin, +50% over latency-greedy

Reinforcement Learning Benefit

- RL enables principled decision-making under instability and partial observability
- Adapts policies online based on real-time feedback

Thank you!